

Definisjon av elementer av typer $T \rightarrow T$ funksjoner

[kap.4-5]

- ▶ Lister: $[typeE]$
- ▶ Produkt (kartesisk): $(typeV, typeH)$
- ▶ Funksjoner: $typeK \rightarrow typeM$
- ▶ Syntaks for typeuttrykk:

$T ::= \text{navn} \mid [T] \mid (\{T, \} T, T) \mid T \rightarrow T \mid (T \rightarrow T)$

$\text{navn} ::= \text{Int} \mid \text{Bool} \mid \dots \mid \text{Var} \mid \text{Brukerdefinert}$

- ▶ betingede likninger:
og $x \ y = \text{if } x == \text{False} \text{ then False} \quad (\text{alltid må ha 'else' gren!})$
 $\text{else if } y == \text{False} \text{ then False}$
 else True
- ▶ voktede likninger:
og $x \ y \mid x == \text{False} = \text{False}$
 $\mid y == \text{False} = \text{False}$
 $\mid \text{otherwise} = \text{True}$
- ▶ anonyme funksjoner:

Mønstre

- ▶ Mønstre er sammensatt av konstruktører som (,), [], :
- ▶ Konstruktører av egen-definerte datatype kan også brukes
- ▶ Generelt format:
f <mønster_1> = <uttrykk_1>
f <mønster_2> = <uttrykk_2>
...
f <mønster_n> = <uttrykk_n>
- ▶ Noen funksjonelle språk forbyr overlapp i mønstre,
andre bruker prinsippet: første mønster til mølla ... Hskl
- ▶ Noen tillater en variabel som det siste mønsteret (overlap!) Hskl
- ▶ Alle krever at variablene innenfor ett og samme mønster er forskjellige Hskl
- ▶ Variablene i <uttrykk_i> er bundet av <mønster_i> Hskl

Lister i Haskell

- ▶ Lister i FP vs. tabeller i IP
Gitt en type A har vi en type $[A]$
- ▶ Rekursjon i FP vs. løkker i IP
En liste er enten tom, $[]$, eller $\dots\dots$ altså **basistilfelle**, og
en verdi etterfulgt av en kortere liste **rekursjonstilfelle**
- ▶ Idéen brukes også til andre induktive datastrukturer, f.eks.:
`data Tre = Leaf Int | Node Int Tre Tre`

noen liste-funksjoner

- ▶ `length :: [a] -> Int`
`head, last :: [a] -> a`
`xs !! n` – n-te elementet i listen `xs`
`tail, init :: [a] -> [a]`
`take n xs` – ta n første elementer fra listen `xs`
`take 2 [1,2,3,4,5] = [1,2]`
`drop n xs` – fjern n første elementer fra listen
`drop 2 [1,2,3,4,5] = [3,4,5]`
`splitAt n xs` – del listen i to, etter n-te elementet
`splitAt 2 [1,2,3,4,5] = ( [1,2], [3,4,5] )`
`splitAt :: Int -> [a] -> ([a], [a])`
$$\text{splitAt } n \text{ } xs = (\text{take } n \text{ } xs, \text{drop } n \text{ } xs)$$
- ▶ `filter my-test xs` – bevar kun el. som tilfredstiller `my-test`
`filter (>5) [2,4,5,6,7] = [6,7]`
`filter odd [2,4,5,6,7] = [5,7]`
`filter :: (a -> Bool) -> [a] -> [a]`

Liste-komprehensjon

- ▶ Mengde-komprehensjon; gitt en mengde M : $\{x \in M \mid P(x)\}$,
 $\{f(x) \mid x \in M \wedge P(x)\}$
- ▶ Liste-komprehensjon, eksempel 1: `[x*x | x <- [1..10] ]`
| leses som 'slik at', `x <- [1..10]` heter generator

eks2: `[x | x <- [1..10], x 'mod' 2 /= 0]`
`x 'mod' 2 /= 0` heter vakt (guard)

eks3: `[(x,y) | x <- [1..3], y <- ['a','b']]`

eks4: `[(x,y) | y <- ['a','b'], x <- [1..3]]`

eks5: `[(x,y) | x <- [1..5], y <- [1..x]]`

eks6 `x = [(xz,y) | y <- [1..x], xz <- [1..5]]`

eks7: `factors n = [x | x <- [1..n], n 'mod' x == 0]`

Flere eksempler

- `factors n = [x | x <- [1..div n 2], n 'mod' x == 0 ] ++ [n]`
- ▶ `prime n = factors n == [1,n]`
 - ▶ `primes n = [x | x <- [1..n], prime x]`
 - ▶ `flatten ll = [e | l <- ll, e <- l]`
`flatten :: [ [t] ] -> [ t ]`
 - ▶ `firsts ps = [x | (x,_) <- ps]` (`_` er en 'wildcard')
`firsts :: [ ( t, t1 ) ] -> [ t ]`
 - ▶ `lookup k ob = [v | (k',v) <- ob, k==k']` (linært søk)
`lookup :: Eq a => a -> [ (a,t) ] -> [t]`
 - ▶ `qs [] = []`
`qs(x:xs) = qs [y| y<-xs, y<=x] ++ [x] ++ qs [y| y<-xs, y>x]`
 - ▶ trekant-tall (triangular numbers): $T_n = \sum_{i=1}^{i=n} i$
 - `tre 1 = 1`
`tre n = n + (tre (n-1))`
 - `tre n = sum [ x | x <- [1..n] ][1..n]` `tre n = (n+1)*n 'div' 2`

Glidelås funksjonen (zip)

`zip` parer elementer i to lister (n . med n ., enn så lenge)

```
zip [] ys = []
```

```
zip xs [] = []
```

```
zip (x:xs) (y:ys) = (x,y) : zip xs ys
```

f.eks: `pairs l = zip l (tail l)`

```
sorted l = and [x <= y | (x,y) <- pairs l]
```

```
sorted' l = null [(x,y) | (x,y) <- pairs l, x>y]
```

f.eks: `posilist :: Eq a => a -> [a] -> [Int]`

```
> posilist 'a' "cabadda" == [2,4,7]
```

```
> posilist 1 [1,1,3,4,1,6] == [1,2,5]
```

```
posilist x xs =
```

```
  [i | (x',i) <- zip xs [1..length xs], x==x']
```

`del` listen i to, med elementer fra tilsv. odde og jevne posisjoner

```
del xs = let pp = (zip xs [1..]) in
```

```
  ([x | (x,i)<-pp, odd i], [x | (x,i)<-pp, even i])
```

Strenger i Haskell

- ▶ Strenger i Haskell er lister av tegn
 - Eksempler: `‘‘abc’’ == ['a','b','c']`, `‘’’ == []`
 - ▶ Liste-funksjoner og -komprehensjon kan brukes for strenger
 - Eksempler: `length ‘‘INF121’’`, `take 3 ‘‘INF121’’...`
 - `antallsifre cs = length [c | c <- cs, isDigit c]`
 - `lower s = [ toLower b | b <- s ]` `import Data.Char`
 - ▶ passord må ha minst 5 små, 3 store bokstaver, og 2 sifre:

```
lud ps = [length [x | x<-ps, isLower x],  
          length [x | x<-ps, isUpper x],  
          length [x | x<-ps, isDigit x] ]
```
- `sjekk ps l u d = [l,u,d] < lud ps` **`[5,3,2] < [6,1,1]`**
`and [e<=i | (e,i) <- zip [l,u,d] (lud ps) ]`

Strenger i Haskell

har hver 'b' i en streng "sin" 'a' foran?

aab "abba" = False

aab "aabCbaDD" = True

aab "abab" = True

aab "abababbCa" = False

aa = [1,3,5,9] bb = [2,4,6,7]

[e<i | (e,i) <- [(1,2),(3,4),(5,6),(9,7)]]

1. aab st = aux st []

aux [] as = True

aux ('a':xs) as = aux xs ('a':as)

aux ('b':xs) [] = False

aux ('b':xs) (a:as) = aux xs as

aux (x:xs) as = aux xs as

2. aab st = let aa = posilist 'a' st

bb = posilist 'b' st in

length aa >= length bb &&

and [e<i | (e,i) <- zip aa bb]

► Hutton, Kap. 5.5: anbefalt lesestoff!

Litt mer strenger (leksikalsk analyse)

tokenize :: String -> [String]

f.eks. "012 * (3 + 11)" må deles i meningsfulle **del**strenger (tokens)

dvs., ["012", "*", "(", "3", "+", "11", ")"]

► tokenize [] = []

tokenize (' ':xs) = tokenize xs

tokenize '('':xs) = "(" : tokenize xs

tokenize (')':xs) = ")" : tokenize xs

tokenize (*':xs) = "*" : tokenize xs

tokenize ('+':xs) = "+" : tokenize xs

tokenize (x:xs) = if isDigit x then –
samle alle siffrerne inntil noe som ikke er et siffer...

(takeWhile isDigit (x:xs)) : tokenize (dropWhile isDigit xs)

tokenize ('i':'f':xs) = "if " : tokenize xs

OSV. ...

tokenize :: String -> String -> String -> [String]

"012 * (3 + 11)" $\mapsto$ ["012", "*", "(", "3", "+", "11", ")"]

... eller lag en liste med enkle tegn som **er** tokens: $t = "*+()"$
og en (disjunkt) liste med enkle tegn som **skiller** tokens: $s = [' ']$

tokenize [] t s = []

tokenize (x:xs) t s

| elem x t = [x] : tokenize xs t s

| elem x s = tokenize xs t s

| otherwise = (takeWhile (....???....))(notin (t++s)) (x:xs))

:

tokenize (dropWhile (notin (t++s)) (x:xs)) t s

der notin xs = \x -> not(elem x xs)